

Tổng Hợp Kiến Thức

1. Metadata

Khái niệm

- **Metadata:** Siêu dữ liệu, thông tin ẩn trong phần `<head>` của trang web, hỗ trợ công cụ tìm kiếm và mạng xã hội.
- **Ai đọc?**
 - Công cụ tìm kiếm (Google, Bing): Sử dụng `title` và `description` để hiển thị kết quả tìm kiếm.
 - Mạng xã hội (Facebook, Zalo): Dùng thẻ Open Graph để tạo bản xem trước (preview).
- **Ví dụ HTML:**

```
<head>
  <title>LetDiv Blog</title>
  <meta name="description" content="Đọc bài viết chi tiết về Tối ưu SEO với Next.js 15">
</head>
```

Static Metadata (Metadata tĩnh)

- **Dùng cho:** Trang có URL tĩnh (trang chủ, giới thiệu).
- **Cấu hình:**
 - Mặc định toàn trang tại `src/app/layout.js` :

```
export const metadata = {
  title: 'LetDiv Blog',
  description: 'Blog chia sẻ kiến thức về lập trình và công nghệ mới nhất.',
};
```

- Trang cụ thể (VD: Trang chủ) tại `src/app/page.js` :

```
export const metadata = {
  title: 'Trang chủ',
  description: 'Đây là trang chủ của LetDiv Blog.',
};
```

- **Kiểm tra:** Dùng "View Page Source" để thấy thẻ `<title>` và `<meta name="description">` .

Dynamic Metadata (Metadata động)

- **Dùng cho:** Trang có URL động (VD: trang chi tiết bài viết).
- **Ví dụ:**
 - API mô phỏng tại `src/app/api/posts/[slug]/route.js` :

```
const posts = [
  { slug: 'bai-viet-1', title: 'Bài viết đầu tiên', description: 'Đây là nội dung của bài viết đầu tiên.' },
  // ...
];
export async function GET(request, { params }) {
  const post = posts.find(p => p.slug === params.slug);
  return post ? NextResponse.json(post) : NextResponse.json({ message: 'Post not found' }, { status: 404 });
}
```

- Trang chi tiết tại `src/app/posts/[slug]/page.jsx` :

```

export async function generateMetadata({ params }) {
  const res = await fetch(`http://localhost:3000/api/posts/${params.slug}`);
  const post = await res.json();
  return post ? { title: post.title, description: `Đọc bài viết chi tiết về ${post.title}` } : { title: 'Không tìm thấy bài viết' };
}

export default async function PostPage({ params }) {
  const res = await fetch(`http://localhost:3000/api/posts/${params.slug}`);
  const post = await res.json();
  return post ? (
    <div>
      <h1>{post.title}</h1>
      <p>{post.description}</p>
      <p>Nội dung bài viết cho slug: {params.slug}</p>
    </div>
  ) : (
    <div>
      <h1>Không tìm thấy bài viết</h1>
      <p>Bài viết bạn đang tìm kiếm không tồn tại.</p>
    </div>
  );
}

```

- **Lưu ý:** Next.js tự động deduplicate `fetch`, chỉ gọi API một lần mỗi render.

2. Hướng dẫn Bot đọc nội dung

Sitemap

- **Khái niệm:** File `sitemap.xml` liệt kê các URL quan trọng, giúp công cụ tìm kiếm khám phá nội dung động.
- **Cách làm:**
 - API mô phỏng tại `src/app/api/posts/route.js`:

```

const posts = [
  { slug: 'bai-viet-1', title: 'Bài viết đầu tiên', lastModified: '2023-01-01T10:00:00Z' },
  // ...
];

export async function GET() {
  return NextResponse.json(posts);
}

```

- Tạo sitemap tại `src/app/sitemap.js`:

```

const BASE_URL = 'http://localhost:3000';
export default async function sitemap() {
  const res = await fetch(`${BASE_URL}/api/posts`);
  const posts = await res.json();
  const postUrls = posts.map((post) => ({
    url: `${BASE_URL}/posts/${post.slug}`,
    lastModified: new Date(post.lastModified),
  }));
  return [
    { url: BASE_URL, lastModified: new Date() },
    { url: `${BASE_URL}/about`, lastModified: new Date() },
    ...postUrls,
  ];
}

```

- **Kiểm tra:** Truy cập `http://localhost:3000/sitemap.xml` .

Robots.txt

- **Khái niệm:** File `robots.txt` chỉ dẫn bot được/không được truy cập phần nào của trang.
- **Tĩnh tại** `app/robots.txt` :

```
User-agent: *
Allow: /
Disallow: /admin/
Sitemap: http://localhost:3000/sitemap.xml
```

- **Động tại** `src/app/robots.js` :

```
const BASE_URL = 'http://localhost:3000';
export default function robots() {
  return {
    rules: {
      userAgent: '*',
      allow: '/',
      disallow: ['/admin/', '/private/'],
    },
    sitemap: `${BASE_URL}/sitemap.xml`,
  };
}
```

3. Tối ưu tài nguyên tĩnh

Tối ưu hình ảnh (`next/image`)

- **Vấn đề:** Thẻ `<img>` gây CLS, không lazy-load, không tối ưu định dạng.
- **Giải pháp:** Dùng `<Image>` :

```
import Image from 'next/image';
<Image src="/next.svg" alt="Sample image" width={250} height={100} />
```

Tối ưu font (`next/font`)

- **Vấn đề:** Tải font qua `<link>` gây chậm, FOUT.
- **Giải pháp:** Dùng `next/font` trong `src/app/layout.js` :

```
import { Geist, Geist_Mono } from "next/font/google";
const geistSans = Geist({ variable: "--font-geist-sans", subsets: ["latin"] });
const geistMono = Geist_Mono({ variable: "--font-geist-mono", subsets: ["latin"] });
```

Tối ưu script (`next/script`)

- **Vấn đề:** Script bên thứ ba chặn render.
- **Giải pháp:** Dùng `<Script>` với các chiến lược:
 - `lazyOnload` : Tải sau khi trang tải xong (Analytics, Chat widgets).
 - `afterInteractive` : Tải sau khi trang tương tác được (Google Tag Manager).
 - `beforeInteractive` : Tải trước tương tác (banner cookie, cần trọng).
- **Ví dụ:**

```
import Script from 'next/script';
<Script src="https://www.googletagmanager.com/gtag/js?id=GA_MEASUREMENT_ID" strategy="lazyOnload" />
```

```
<Script id="google-analytics" strategy="lazyOnload">
{
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', 'GA_MEASUREMENT_ID');
}
</Script>
```

4. Đo lường & Phân tích hiệu suất

Lighthouse

- **Cách chạy:** Chrome DevTools → Tab Lighthouse → "Analyze page load".
- **Điểm số:** Performance, Accessibility, Best Practices, SEO (0-100).

Core Web Vitals

- **LCP (Tốc độ tải):** $\leq 2.5s$, tối ưu bằng `next/image` (`priority`), `next/font` .
- **INP (Tương tác):** $\leq 200ms$, tối ưu bằng Server Components, `next/script` , code splitting.
- **CLS (Ổn định bố cục):** ≤ 0.1 , set `width` , `height` cho `next/image` .

Workflow

1. Chạy Lighthouse trước khi tối ưu (Baseline).
2. Áp dụng kỹ thuật tối ưu.
3. Chạy lại Lighthouse (Measure).
4. So sánh điểm số (Compare).
5. Lặp lại đến khi đạt mục tiêu.